

- Manual de R para Investigación de Operaciones en Ingeniería de Sistemas de Información
 - Tema 1: Introducción a R y RStudio para Ingeniería de Sistemas
 - 1.1 ¿Qué es R y por qué es esencial para la Ingeniería en Sistemas de Información?
 - 1.2 Instalación de R y RStudio: Configuración del entorno profesional
 - 1.3 Interface de RStudio: Herramientas para el ingeniero de sistemas
 - 1.4 Primeros pasos: Operaciones básicas aplicadas
 - 1.5 Problema integrador: Análisis inicial de capacidad de servidor
 - 1.6 Ejercicios de autoevaluación
 - 1.7 Recursos adicionales y referencias
 - Tema 2: Fundamentos de Programación en R para Análisis de Sistemas
 - 2.1 Introducción a la Programación en R
 - 2.2 Variables y Tipos de Datos Básicos
 - 2.3 Operadores Básicos
 - 2.4 Estructuras de Datos Básicas
 - 2.5 Funciones Básicas
 - 2.6 Estructuras de Control
 - 2.7 Creando Funciones Propias
 - 2.8 Trabajando con Archivos CSV
 - 2.9 Ejercicios Prácticos
 - 2.10 Consejos para Principiantes
 - Tema 3: Manipulación de Datos con dplyr y tidyr para Análisis de Sistemas
 - 3.1 Introducción a la Manipulación de Datos
 - Filosofía de los paquetes tidyverse
 - Instalación y carga de paquetes
 - 3.2 Importación de Datos desde Archivos CSV
 - La importancia de una buena importación de datos
 - Creación y lectura de un archivo CSV de ejemplo
 - 3.3 Verbos Básicos de dplyr
 - 3.3.1 filter() - Filtrar filas basado en condiciones
 - 3.3.2 select() - Seleccionar columnas específicas
 - 3.3.3 mutate() - Crear nuevas columnas
 - 3.3.4 arrange() - Ordenar filas
 - 3.3.5 summarize() - Resumir datos
 - 3.4 Operaciones con group_by()
 - El poder del agrupamiento en el análisis de datos

- 3.5 Manipulación de Datos con tidyr
 - 3.5.1 pivot_longer() - Convertir datos wide a long
 - 3.5.2 pivot_wider() - Convertir datos long a wide
 - 3.5.3 separate() - Dividir columnas
- 3.6 Unión de Datos
 - Combinando información de múltiples fuentes
- 3.7 Ejercicio Práctico: Análisis de Rendimiento Completo
- 3.8 Ejercicio Avanzado: Simulación de Datos y Análisis
- 3.9 Buenas Prácticas y Consejos
- 3.10 Recursos Adicionales

Manual de R para Investigación de Operaciones en Ingeniería de Sistemas de Información

Tema 1: Introducción a R y RStudio para Ingeniería de Sistemas

1.1 ¿Qué es R y por qué es esencial para la Ingeniería en Sistemas de Información?

Problema: Imagine que debe analizar el rendimiento de un servidor web bajo diferentes cargas de usuarios. ¿Cómo procesaría miles de registros de log, calcularía métricas de performance y visualizaría los resultados para tomar decisiones de capacidad?

Solución con R: R es un lenguaje de programación especializado en análisis estadístico y visualización que permite manipular grandes volúmenes de datos, implementar algoritmos complejos y generar reportes reproducibles - exactamente lo que necesita un ingeniero de sistemas.

Según Chambers (2008) en "*Software for Data Analysis*", R proporciona un entorno unificado para manipulación, cálculo y visualización de datos que es

especialmente valioso para problemas de ingeniería de sistemas donde los datos son multidimensionales y requieren transformaciones complejas.

Aplicación en sistemas de información e investigación de operaciones:

- Análisis de rendimiento de sistemas
 - Simulación de carga de usuarios
 - Optimización de recursos computacionales
 - Predicción de fallos y capacidad
 - Análisis de redes y conectividad
-

1.2 Instalación de R y RStudio: Configuración del entorno profesional

Instalación de R:

Para usuarios de Windows:

Visitar [R para windows](#) Descargar y ejecutar el instalador

Para Ubuntu/Debian:

```
sudo apt-get install r-base r-base-dev
```

Para macOS:

Visitar [R para mac](#)

Instalación de RStudio:

- Descargar desde [R Studio](#)
- Instalar la versión Desktop Open Source License

Verificación de la instalación:

Ejecutar en la consola de R:

```
version$version.string
```

Debería mostrar: *"R version 4.3.1 (2023-06-16)"*

Según Wickham & Grolemund (2017) en *"R for Data Science"*, es fundamental mantener un entorno consistente mediante el uso de proyectos específicos y control de versiones.

1.3 Interface de RStudio: Herramientas para el ingeniero de sistemas

Páneos principales y su utilidad en ingeniería de sistemas:

- **Consola (Console):** Ejecución inmediata de código y pruebas rápidas

Ejemplo: Calcular capacidad teórica de servidor

```
capacidad_teorica <- function(usuarios_concurrentes, tiempo_respuesta) {  
  return(usuarios_concurrentes / tiempo_respuesta)  
}  
capacidad_teorica(1000, 2) # 500 transacciones/segundo
```

- **Editor de scripts (Source):** Donde se desarrollan programas completos

Ejemplo: Script para análisis de rendimiento

```
analizar_rendimiento <- function(datos_servidor) {  
  uso_cpu <- mean(datos_servidor$cpu_usage)  
  latency_p95 <- quantile(datos_servidor$response_time, 0.95)  
  return(list(cpu = uso_cpu, latency = latency_p95))  
}
```

- **Entorno (Environment):** Monitorización de variables y objetos en memoria
Fundamental para debuggear problemas de rendimiento y memoria
- **Panel de gráficos (Plots):** Visualización de métricas de performance

Ejemplo rápido de visualización:

```
load_data <- c(65, 78, 82, 90, 95, 98, 100, 99, 97)  
plot(load_data, type = "b", main = "Carga del servidor bajo estrés")
```

1.4 Primeros pasos: Operaciones básicas aplicadas

Problema: Calcular métricas de performance básicas para un servidor web.

Operaciones fundamentales:

1. Asignación de variables (asignando métricas de performance)

```
response_times <- c(2.1, 1.8, 2.5, 3.2, 2.9, 1.5, 2.8)
concurrent_users <- c(100, 150, 200, 250, 300, 350, 400)
```

2. Cálculo de estadísticas descriptivas (según Jain, 1991 en *"The Art of Computer Systems Performance Analysis"*)

```
mean_response <- mean(response_times)
median_response <- median(response_times)
std_dev_response <- sd(response_times)
```

4. Análisis de correlación entre usuarios y tiempo de respuesta

```
correlation <- cor(concurrent_users, response_times)
```

5. Resultados

```
cat("Estadísticas de rendimiento:\n")
cat("Tiempo promedio de respuesta:", mean_response, "segundos\n")
cat("Desviación estándar:", std_dev_response, "\n")
cat("Correlación usuarios-tiempo:", correlation, "\n")
```

Sistema de ayuda integrado:

```
?mean
?sd
?cor
```

Buscar por tema:

```
??"standard deviation"  
??"correlation"
```

1.5 Problema integrador: Análisis inicial de capacidad de servidor

Escenario: Usted es ingeniero de sistemas en una empresa que experimenta problemas de rendimiento. Debe analizar datos preliminares para determinar si existe una relación entre la cantidad de usuarios concurrentes y el tiempo de respuesta.

Solución paso a paso:

Datos simulados (en la práctica vendrían de logs reales):

```
usuarios <- c(50, 100, 150, 200, 250, 300, 350, 400)  
tiempos_respuesta <- c(1.2, 1.5, 1.8, 2.1, 2.7, 3.5, 4.8, 6.3)
```

1. Análisis exploratorio básico

```
summary(tiempos_respuesta)  
cat("Máximo tiempo de respuesta:", max(tiempos_respuesta), "segundos\n")
```

2. Modelado preliminar (regresión lineal simple)

```
modelo <- lm(tiempos_respuesta ~ usuarios)  
coeficientes <- coef(modelo)
```

3. Predicción para 500 usuarios

```
prediccion_500 <- coeficientes[1] + coeficientes[2] \* 500  
cat("Tiempo de respuesta estimado para 500 usuarios:", round(prediccion_500, 2),  
"segundos\n")
```

4. Evaluación de capacidad según SLA (Service Level Agreement)

```
sla_maximo <- 5.0 # 5 segundos máximo de respuesta
capacidad_maxima <- (sla_maximo - coeficientes[1]) / coeficientes[2]
cat("Capacidad máxima dentro del SLA:", round(capacidad_maxima), "usuarios
concurrentes\n")
```

1.6 Ejercicios de autoevaluación

Ejercicio 1: Calcule el percentil 95 de los tiempos de respuesta y determine si cumple con un SLA de 4 segundos para el 95% de las requests.

Datos:

```
response_times <- c(1.2, 1.5, 1.8, 2.1, 2.7, 3.5, 4.8, 6.3, 2.9, 3.1)
```

Solución:

```
p95 <- quantile(response_times, 0.95)
cumple_sla <- p95 <= 4
cat("Percentil 95:", p95, "\n")
cat("¿Cumple SLA?", cumple_sla, "\n")
```

Ejercicio 2: Simule la carga de usuarios para una hora pico y calcule los recursos necesarios.

Parámetros del sistema:

```
usuarios_pico <- 1000
transacciones_por_usuario <- 5
tiempo_promedio_transaccion <- 2.5 # segundos
```

Cálculo de capacidad requerida:

```
total_transacciones <- usuarios_pico * transacciones_por_usuario
capacidad_requerida <- total_transacciones / tiempo_promedio_transaccion
cat("Capacidad requerida:", capacidad_requerida, "transacciones/segundo\n")
```

1.7 Recursos adicionales y referencias

Libro recomendado: *"The Art of Computer Systems Performance Analysis"* by Raj Jain

Paquetes R útiles para ingeniería de sistemas:

- bench para benchmarking de código
- profvis para profiling de performance
- future para procesamiento paralelo

Comunidades: RStudio Community, Stack Overflow para preguntas específicas

"Without data, you're just another person with an opinion" - W. Edwards Deming, estadístico y pionero en control de calidad, cuya filosofía es fundamental para la ingeniería de sistemas basada en datos.

Tema 2: Fundamentos de Programación en R para Análisis de Sistemas

2.1 Introducción a la Programación en R

Para un ingeniero de sistemas, la programación es fundamental para:

- **Automatización de tareas:** Permite automatizar procesos repetitivos como monitoreo de sistemas.
- **Análisis de datos:** Facilita el análisis de grandes volúmenes de datos de sistemas.
- **Simulación:** Permite modelar y simular el comportamiento de sistemas antes de implementarlos.
- **Resolución de problemas:** Ayuda a resolver problemas complejos mediante algoritmos.

R es especialmente útil porque está diseñado específicamente para análisis de datos, tiene una amplia comunidad de usuarios, es gratuito y cuenta con herramientas poderosas para visualización.

2.2 Variables y Tipos de Datos Básicos

¿Qué es una variable?

Una variable es como un contenedor con una etiqueta donde puedes almacenar información. La etiqueta es el nombre de la variable y el contenido es el valor.

Ejemplos:

- Variable numérica (almacena números):

```
cantidad_servidores <- 5
temperatura_cpu <- 65.5
```

- Variable de texto (entre comillas):

```
nombre_servidor <- "servidor_principal"
ubicacion <- "Sala A"
```

- Variable lógica (TRUE o FALSE):

```
esta_activo <- TRUE
necesita_mantenimiento <- FALSE
```

Tipos de datos comunes en R:

Tipo	Ejemplo	Descripción	Uso en sistemas
numeric	15.7	Números con decimales	Métricas de rendimiento
integer	10L	Números enteros	Cantidad de dispositivos
character	"Hola"	Texto	Nombres de servidores
logical	TRUE	Verdadero o falso	Estados de conexión

2.3 Operadores Básicos

Operadores aritméticos:

- Suma: `usuarios_concurrentes <- 100 + 50`
- Resta: `espacio_libre <- 1000 - 350`
- Multiplicación: `almacenamiento_total <- 500 * 4`
- División: `promedio_uso <- 450 / 5`
- Potencia: `bytes_en_gb <- 1024^3`

Operadores de comparación:

- Igualdad: `5 == 5 → TRUE`
 - Desigualdad: `5 != 3 → TRUE`
 - Mayor que: `10 > 5 → TRUE`
 - Menor que: `3 < 2 → FALSE`
 - Mayor o igual: `5 >= 5 → TRUE`
-

2.4 Estructuras de Datos Básicas

Vectores: listas de valores del mismo tipo.

Ejemplo:

```
tiempos_respuesta <- c(2.1, 1.8, 2.5, 3.2, 2.9)
```

Data frames: tablas con filas y columnas.

Ejemplo:

```
recursos_sistema <- data.frame(  
  servidor = c("srv01", "srv02", "srv03"),  
  ram_gb = c(16, 32, 64),  
  cpu_cores = c(4, 8, 16),  
  activo = c(TRUE, TRUE, FALSE)  
)
```

2.5 Funciones Básicas

Una función es como una herramienta que realiza una tarea específica.

Ejemplos en R:

- `mean()` → promedio
 - `max()` → valor máximo
 - `min()` → valor mínimo
 - `length()` → número de elementos
 - `summary()` → resumen estadístico
-

2.6 Estructuras de Control

Condicionales:

Permiten tomar decisiones.

Ejemplo:

```
if (uso_cpu > 90) { ... } else { ... }
```

Bucles:

Repiten acciones automáticamente.

Ejemplo: `for (hora in 1:8) { ... }`

2.7 Creando Funciones Propias

Ejemplo:

```
calcular_capacidad <- function(usuarios, recursos_por_usuario, recursos_totales) {  
  usuarios_soportados <- recursos_totales / recursos_por_usuario  
  if (usuarios <= usuarios_soportados) {  
    mensaje <- paste("Sistema puede soportar", usuarios, "usuarios. Capacidad  
adecuada.")  
  } else {  
    mensaje <- paste("ALERTA: Solo soporta", floor(usuarios_soportados),  
"usuarios.")  
  }  
  return(mensaje)  
}
```

2.8 Trabajando con Archivos CSV

- Guardar datos: `write.csv(datos, "archivo.csv", row.names = FALSE)`
- Leer datos: `read.csv("archivo.csv")`

Ejemplo de aplicación: guardar métricas de servidores y luego analizarlas.

2.9 Ejercicios Prácticos

Ejercicio 1: Monitoreo Simple de Sistema

Analizar uso promedio de memoria y disco y generar un informe CSV con resultados.

Ejercicio 2: Análisis de Rendimiento de Servidor Web

Calcular estadísticas básicas de tiempos de respuesta, verificar cumplimiento de estándar y generar gráfico comparativo.

2.10 Consejos para Principiantes

- Empieza simple y aumenta la complejidad poco a poco.
 - Prueba el código frecuentemente.
 - Usa comentarios para documentar.
 - No memorices: usa la ayuda de R.
 - Practica regularmente.
 - Aprende de los errores.
-

Tema 3: Manipulación de Datos con dplyr y tidyr para Análisis de Sistemas

3.1 Introducción a la Manipulación de Datos

¿Qué es la manipulación de datos y por qué es crucial para ingenieros de sistemas?

La manipulación de datos es el proceso de limpiar, transformar y reorganizar datos brutos para hacerlos adecuados para el análisis. Como ingeniero de sistemas, te encontrarás constantemente con datos de diversas fuentes: logs de servidores, métricas de rendimiento, registros de red, y más. Estos datos rara vez vienen en el formato perfecto para análisis inmediato.

dplyr y **tidyr** son paquetes de R diseñados específicamente para estas tareas. Forman parte del "tidyverse", una colección de paquetes de R para ciencia de datos que comparten una filosofía común de organización de datos y gramática para la manipulación de datos.

Filosofía de los paquetes tidyverse

La filosofía detrás de dplyr y tidyr se basa en tres principios fundamentales:

- Cada variable forma una columna
- Cada observación forma una fila
- Cada tipo de unidad observacional forma una tabla

Estos principios crean datos "tidy" (ordenados), que son mucho más fáciles de manipular, visualizar y modelar.

Instalación y carga de paquetes

```
# Instalar los paquetes necesarios
# Esto solo necesitas hacerlo una vez en tu computadora
install.packages("dplyr")    # Para manipulación de datos
install.packages("tidyr")    # Para reorganización de datos
install.packages("readr")    # Para lectura eficiente de archivos

# Cargar los paquetes en tu sesión de R
# Esto debes hacerlo cada vez que inicies R y quieras usar estos paquetes
library(dplyr) # Carga las funciones de dplyr
library(tidyr) # Carga las funciones de tidyr
library(readr) # Carga las funciones de readr

# Verificar que los paquetes se cargaron correctamente
# No debería aparecer ningún mensaje de error
```

3.2 Importación de Datos desde Archivos CSV

La importancia de una buena importación de datos

En ingeniería de sistemas, frecuentemente trabajarás con datos exportados desde sistemas de monitoreo, logs de servidores, o herramientas de análisis de rendimiento. Estos datos comúnmente se exportan en formato CSV (Comma-Separated Values).

La función `read_csv()` del paquete `readr` es más eficiente que la función base `read.csv()` porque:

- Es más rápida con archivos grandes
- No convierte strings a factores por defecto
- Muestra una barra de progreso para archivos grandes
- Proporciona mejores mensajes de error

Creación y lectura de un archivo CSV de ejemplo

```
# Primero, creemos un conjunto de datos de ejemplo que simule métricas de
servidores

# Este data frame representa mediciones tomadas cada 5 minutos de diferentes
servidores

datos_ejemplo <- data.frame(
  servidor = c("web01", "web02", "db01", "db02", "cache01", "web03"), # Nombres de
servidores
  cpu_uso = c(45, 62, 38, 71, 56, 48), # % de uso de CPU
  mem_uso = c(60, 75, 45, 82, 67, 58), # % de uso de memoria
  temp = c(65, 68, 62, 72, 66, 64), # Temperatura en °C
  estado = c("activo", "activo", "activo", "sobrecarga", "activo", "activo"), #
Estado del servidor
  timestamp = as.POSIXct(c("2023-10-01 08:00:00", "2023-10-01 08:05:00",
"2023-10-01 08:10:00", "2023-10-01 08:15:00",
"2023-10-01 08:20:00", "2023-10-01 08:25:00")) # Marca de tiempo
)

# Guardar el data frame en un archivo CSV

# write_csv() es la función de readr para escribir archivos CSV

write_csv(datos_ejemplo, "datos_servidores.csv")

# Ahora leemos el archivo CSV que acabamos de crear

# read_csv() detecta automáticamente los tipos de datos

datos_servidores <- read_csv("datos_servidores.csv")

# Examinar la estructura de los datos importados

# glimpse() proporciona una vista transposed de los datos (filas → columnas,
columnas → filas)

print("Estructura de los datos importados:")
glimpse(datos_servidores)

# También podemos usar head() para ver las primeras filas
```

```
print("Primeras filas de los datos:")
head(datos_servidores)
```

3.3 Verbos Básicos de dplyr

dplyr proporciona una gramática coherente para la manipulación de datos, con verbos que corresponden a las operaciones más comunes. El operador %>% (pipe) nos permite encadenar estas operaciones de manera legible.

3.3.1 filter() - Filtrar filas basado en condiciones

El verbo filter() permite seleccionar un subconjunto de filas basado en condiciones lógicas. Es especialmente útil para focalizar el análisis en datos relevantes.

```
# Filtrar servidores con uso de CPU mayor a 50%

# %>% pasa el resultado de la izquierda a la función de la derecha

servidores_alta_carga <- datos_servidores %>%
  filter(cpu_uso > 50) # Mantiene solo filas donde cpu_uso > 50

print("Servidores con alta carga de CPU (mayor al 50%):")
print(servidores_alta_carga)

# Filtrar servidores activos con temperatura menor a 70°C

# Podemos combinar múltiples condiciones con , o &

servidores_estables <- datos_servidores %>%
  filter(estado == "activo", # Estado debe ser "activo"
        temp < 70) # Temperatura menor a 70°C

print("Servidores estables (activos y temperatura < 70°C):")
print(servidores_estables)

# Filtrar usando operador OR (|)

servidores_problematicos <- datos_servidores %>%
  filter(cpu_uso > 70 | # Uso de CPU > 70%
        mem_uso > 75 | # Uso de memoria > 75%
        temp > 70) # Temperatura > 70°C

print("Servidores con al menos un indicador problemático:")
print(servidores_problematicos)
```

3.3.2 select() - Seleccionar columnas específicas

El verbo select() permite elegir un subconjunto de columnas, útil cuando trabajas con datasets con muchas columnas y solo necesitas algunas.

```
# Seleccionar solo las columnas de interés para análisis de rendimiento

# Listamos los nombres de las columnas que queremos mantener

metricas_esenciales <- datos_servidores %>%
  select(servidor, # Columna con nombre del servidor
         cpu_uso, # Columna con uso de CPU
         mem_uso, # Columna con uso de memoria
         temp) # Columna con temperatura

print("Métricas esenciales de rendimiento:")
print(metricas_esenciales)

# Seleccionar columnas que contengan "uso" en su nombre

# contains() es una función helper que busca patrones en nombres de columnas

metricas_uso <- datos_servidores %>%
  select(servidor, # Mantener columna servidor
         contains("uso")) # Mantener columnas que contengan "uso"

print("Métricas de uso (CPU y memoria):")
print(metricas_uso)

# También podemos excluir columnas usando el signo -

metricas_sin_tiempo <- datos_servidores %>%
  select(-timestamp) # Excluir columna timestamp

print("Datos sin la columna de timestamp:")
print(metricas_sin_tiempo)
```

3.3.3 mutate() - Crear nuevas columnas

El verbo mutate() permite crear nuevas columnas basadas en transformaciones de columnas existentes. Es extremadamente útil para calcular métricas derivadas.

```
# Crear nuevas columnas calculadas basadas en las existentes

datos_analisis <- datos_servidores %>%
  mutate( # Calcular si el servidor está bajo estrés (condición lógica) # Un servidor
         # está bajo estrés si uso de CPU > 70% O uso de memoria > 75%
         bajo_estres = cpu_uso > 70 | mem_uso > 75,
```



```

# Categorizar nivel de temperatura usando case_when()
# case_when() permite crear condiciones complejas similares a if-else
nivel_temp = case_when(
  temp < 65 ~ "Óptima",           # Si temp < 65 → "Óptima"
  temp >= 65 & temp < 70 ~ "Aceptable", # Si 65 ≤ temp < 70 → "Aceptable"
  temp >= 70 ~ "Crítica"         # Si temp ≥ 70 → "Crítica"
),

# Calcular un índice de salud del servidor (fórmula compuesta)
# Pondera diferentes métricas para crear un índice único
indice_salud = 100 - (cpu_uso * 0.4 + # CPU contribuye 40%
                     mem_uso * 0.3 + # Memoria contribuye 30%
                     temp / 2)       # Temperatura contribuye (temp/2)%

)

print("Datos con nuevas métricas calculadas:")
print(datos_analisis)

# Podemos usar mutate() para modificar columnas existentes también

datos_normalizados <- datos_servidores %>%
mutate( # Normalizar métricas a escala 0-1
  cpu_normalizado = cpu_uso / 100, # Convertir % a proporción
  mem_normalizado = mem_uso / 100 # Convertir % a proporción
)

print("Datos con métricas normalizadas:")
print(datos_normalizados)

```

3.3.4 arrange() - Ordenar filas

El verbo `arrange()` ordena las filas de un data frame basado en los valores de una o más columnas. Por defecto ordena en orden ascendente.

```

# Ordenar por uso de CPU descendente

# desc() ordena en orden descendente

servidores_ordenados_cpu <- datos_servidores %>%
  arrange(desc(cpu_uso)) # Ordenar de mayor a menor uso de CPU

print("Servidores ordenados por uso de CPU (descendente):")
print(servidores_ordenados_cpu)

# Ordenar por múltiples criterios

# Primero por estado, luego por uso de memoria descendente

servidores_ordenados_multiple <- datos_servidores %>%
  arrange(estado, # Ordenar por estado (A-Z)

```

```

desc(mem_uso)) # Luego por memoria descendente

print("Servidores ordenados por estado y uso de memoria:")
print(servidores_ordenados_multiple)

# Ordenar personalizado con factor levels

# Podemos definir un orden específico para una columna

orden_estados <- c("sobrecarga", "activo") # Definir orden deseado

datos_ordenados_personalizado <- datos_servidores %>%
mutate(estado = factor(estado, levels = orden_estados)) %>% # Convertir a factor
con orden específico
arrange(estado) # Ordenar según el orden del factor

print("Servidores ordenados por estado personalizado:")
print(datos_ordenados_personalizado)

```

3.3.5 summarize() - Resumir datos

El verbo `summarize()` (o `summarise()`) colapsa múltiples valores en un solo valor resumen. Es especialmente útil con `group_by()` para crear resúmenes por grupo.

```

# Calcular estadísticas resumidas para todo el dataset

resumen_estadisticas <- datos_servidores %>%
summarize(
n_servidores = n(), # Contar número de filas
cpu_promedio = mean(cpu_uso), # Calcular promedio de CPU
cpu_maximo = max(cpu_uso), # Encontrar valor máximo de CPU
mem_promedio = mean(mem_uso), # Calcular promedio de memoria
temp_promedio = mean(temp), # Calcular promedio de temperatura
servidores_sobrecarga = sum(estado == "sobrecarga") # Contar servidores en
sobrecarga
)

print("Resumen estadístico del sistema:")
print(resumen_estadisticas)

# Podemos calcular múltiples estadísticas para una misma columna

resumen_detallado <- datos_servidores %>%
summarize( # Para uso de CPU
cpu_media = mean(cpu_uso),
cpu_mediana = median(cpu_uso),
cpu_desviacion = sd(cpu_uso),
cpu_min = min(cpu_uso),
cpu_max = max(cpu_uso),

# Para uso de memoria
mem_media = mean(mem_uso),

```

```
mem_mediana = median(mem_uso),

# Para temperatura
temp_media = mean(temp)

)

print("Resumen detallado de métricas:")
print(resumen_detallado)
```

3.4 Operaciones con group_by()

El poder del agrupamiento en el análisis de datos

El verbo `group_by()` permite agrupar datos por una o más variables, para luego realizar operaciones dentro de cada grupo. Esto es extremadamente útil para análisis comparativos.

```
# Agrupar por estado y calcular métricas para cada grupo

metricas_por_estado <- datos_servidores %>%
  group_by(estado) %>% # Agrupar por la columna estado
  summarize(
    n = n(), # Número de servidores en cada estado
    cpu_promedio = mean(cpu_uso), # Promedio de CPU por estado
    mem_promedio = mean(mem_uso), # Promedio de memoria por estado
    temp_promedio = mean(temp) # Promedio de temperatura por estado
  ) %>%
  arrange(desc(cpu_promedio)) # Ordenar por CPU promedio descendente

print("Métricas agrupadas por estado del servidor:")
print(metricas_por_estado)

# Agrupar por tipo de servidor (inferido del nombre)

metricas_por_tipo <- datos_servidores %>%
  mutate( # Extraer tipo de servidor del nombre (primera parte antes de números)
    tipo_servidor = case_when(
      grepl("^web", servidor) ~ "Web Server", # Si empieza con "web"
      grepl("^db", servidor) ~ "Database Server", # Si empieza con "db"
      grepl("^cache", servidor) ~ "Cache Server", # Si empieza con "cache"
      TRUE ~ "Otro" # Para cualquier otro caso
    )
  ) %>%
  group_by(tipo_servidor) %>% # Agrupar por el tipo de servidor
  summarize(
    n = n(), # Número de servidores por tipo
    cpu_promedio = round(mean(cpu_uso), 1), # CPU promedio redondeado
```

```

mem_promedio = round(mean(mem_uso), 1), # Memoria promedio redondeado
temp_maxima = max(temp) # Temperatura máxima por tipo
)

print("Métricas por tipo de servidor:")
print(metrics_por_tipo)

# Agrupamiento múltiple y operaciones más complejas

# Podemos agrupar por múltiples variables y calcular varias métricas

if ("timestamp" %in% colnames(datos_servidores)) {
  metrics_por_hora_estado <- datos_servidores %>%
  mutate(hora = format(timestamp, "%H:%M")) %>% # Extraer hora del timestamp
  group_by(hora, estado) %>% # Agrupar por hora y estado
  summarize(
    n = n(),
    cpu_promedio = mean(cpu_uso),
    .groups = "drop" # Eliminar agrupamiento después del summarize
  )

  print("Métricas por hora y estado:")
  print(metrics_por_hora_estado)
}

```

3.5 Manipulación de Datos con tidyr

tidyr complementa a **dplyr** proporcionando funciones para reorganizar datos, especialmente para cambiar entre formatos wide (ancho) y long (largo).

3.5.1 pivot_longer() - Convertir datos wide a long

Los datos en formato "wide" tienen una columna para cada variable, mientras que los datos en formato "long" tienen una columna para los nombres de variables y otra para los valores. `pivot_longer()` convierte de wide a long.

```

# Crear un subset de datos en formato wide

datos_wide <- datos_servidores %>%
select(servidor, cpu_uso, mem_uso, temp) # Seleccionar columnas de métricas

print("Datos en formato wide (una columna por métrica):")
print(datos_wide)

# Convertir a formato long (más adecuado para visualización y algunos análisis)

datos_long <- datos_wide %>%

```

```

pivot_longer(
  cols = c(cpu_uso, mem_uso, temp), # Columnas a convertir
  names_to = "metrica", # Nombre de la nueva columna para los nombres de métricas
  values_to = "valor" # Nombre de la nueva columna para los valores
)

print("Datos en formato long (una fila por combinación servidor-métrica):")
print(datos_long)

# Podemos hacer análisis más fácilmente con datos en formato long

analisis_long <- datos_long %>%
  group_by(metrica) %>%
  summarize(
    promedio = mean(valor),
    maximo = max(valor),
    minimo = min(valor)
  )

print("Resumen de métricas en formato long:")
print(analisis_long)

```

3.5.2 `pivot_wider()` - Convertir datos long a wide

`pivot_wider()` es la operación inversa a `pivot_longer()`, convirtiendo datos de formato long a wide.

```

# Convertir de vuelta a formato wide desde formato long

datos_wide_nuevo <- datos_long %>%
  pivot_wider(
    names_from = metrica, # Columna que contiene los nombres para las nuevas columnas
    values_from = valor # Columna que contiene los valores para las nuevas columnas
  )

print("Datos convertidos de nuevo a formato wide:")
print(datos_wide_nuevo)

# pivot_wider() es útil para crear tablas resumen

tabla_resumen <- datos_servidores %>%
  group_by(estado) %>%
  summarize(
    cpu_promedio = mean(cpu_uso),
    mem_promedio = mean(mem_uso)
  ) %>%
  pivot_wider(
    names_from = estado,
    values_from = c(cpu_promedio, mem_promedio)
  )

```

```
print("Tabla resumen con métricas por estado:")
print(tabla_resumen)
```

3.5.3 `separate()` - Dividir columnas

El verbo `separate()` divide una columna en múltiples columnas basado en un separador. Útil cuando tienes información combinada en una sola columna.

```
# Crear datos con información combinada (simulando datos de un sistema legacy)

datos_combinados <- data.frame(
  servidor_info = c("web01-salaA-activo", "db01-salaB-activo", "cache01-salaA-
mantenimiento"),
  cpu_uso = c(45, 38, 56)
)

print("Datos con información combinada en una columna:")
print(datos_combinados)

# Separar la columna combinada en columnas individuales

datos_separados <- datos_combinados %>%
  separate(
    servidor_info, # Columna a separar
    into = c("servidor", "sala", "estado"), # Nombres de las nuevas columnas
    sep = "-" # Separador (guión)
  )

print("Datos después de separar la columna combinada:")
print(datos_separados)

# Podemos controlar cuántas divisiones hacer

datos_separados_limitados <- datos_combinados %>%
  separate(
    servidor_info,
    into = c("servidor", "info_restante"), # Solo dos nuevas columnas
    sep = "-", # Separador
    extra = "merge" # Mantener el resto unido en la última columna
  )

print("Datos con separación limitada:")
print(datos_separados_limitados)
```

3.6 Unión de Datos

Combinando información de múltiples fuentes

En situaciones reales, frecuentemente necesitarás combinar datos de múltiples fuentes. dplyr proporciona varias funciones para unir datos:

- `left_join()`: Mantiene todas las filas del dataset izquierdo
- `right_join()`: Mantiene todas las filas del dataset derecho
- `inner_join()`: Mantiene solo filas con coincidencias en ambos datasets
- `full_join()`: Mantiene todas las filas de ambos datasets

```
# Crear un dataset con información adicional de servidores

info_servidores <- data.frame(
  servidor = c("web01", "web02", "db01", "db02", "cache01", "web03"),
  capacidad_ram = c(16, 32, 64, 64, 32, 16), # RAM en GB
  capacidad_cpu = c(4, 8, 16, 16, 8, 4), # Núcleos de CPU
  sistema_operativo = c("Linux", "Linux", "Linux", "Windows", "Linux", "Linux"),
  ubicacion = c("Sala A", "Sala B", "Sala C", "Sala C", "Sala A", "Sala B")
)

print("Información adicional de servidores:")
print(info_servidores)

# Unir los datos de rendimiento con la información de capacidad

# left_join() mantiene todos los servidores del dataset izquierdo
(datos_servidores)

datos_completos <- datos_servidores %>%
  left_join(info_servidores, by = "servidor") %>% # Unir por columna servidor
  mutate( # Calcular porcentaje de uso respecto a la capacidad
    ram_porcentaje = (mem_uso / capacidad_ram) _ 100, # % de RAM utilizada
    cpu_porcentaje = (cpu_uso / capacidad_cpu) _ 100 # % de CPU utilizada
  )

print("Datos completos unidos con información de capacidad:")
print(datos_completos)

# Identificar servidores que no tienen información de capacidad

servidores_sin_info <- datos_servidores %>%
  anti_join(info_servidores, by = "servidor") # Encuentra filas sin coincidencia

print("Servidores sin información de capacidad:")
print(servidores_sin_info)

# Unión con múltiples columnas clave (si fuera necesario)

# Supongamos que necesitamos unir por servidor y timestamp

if (all(c("servidor", "timestamp") %in% colnames(datos_servidores))) {
```

```
# Crear datos adicionales con timestamp

info_detallada <- data.frame(
  servidor = c("web01", "web02"),
  timestamp = as.POSIXct(c("2023-10-01 08:00:00", "2023-10-01 08:05:00")),
  proceso_principal = c("nginx", "apache")
)

# Unir por múltiples columnas

datos_detallados <- datos_servidores %>%
  inner_join(info_detallada, by = c("servidor", "timestamp"))

print("Datos con información detallada por servidor y timestamp:")
print(datos_detallados)
}
```

3.7 Ejercicio Práctico: Análisis de Rendimiento Completo

Vamos a aplicar todo lo aprendido en un análisis completo de rendimiento de sistemas.

```
# Cargar datos de ejemplo (si no están disponibles)

if (!file.exists("datos_servidores.csv")) {

  # Crear datos de ejemplo

  datos_ejemplo <- data.frame(
    servidor = c("web01", "web02", "db01", "db02", "cache01", "web03"),
    cpu_uso = c(45, 62, 38, 71, 56, 48),
    mem_uso = c(60, 75, 45, 82, 67, 58),
    temp = c(65, 68, 62, 72, 66, 64),
    estado = c("activo", "activo", "activo", "sobrecarga", "activo", "activo"),
    timestamp = as.POSIXct(c("2023-10-01 08:00:00", "2023-10-01 08:05:00",
      "2023-10-01 08:10:00", "2023-10-01 08:15:00",
      "2023-10-01 08:20:00", "2023-10-01 08:25:00"))
  )
  write_csv(datos_ejemplo, "datos_servidores.csv")
  print("Archivo de datos creado: datos_servidores.csv")
}

# Leer los datos

datos <- read_csv("datos_servidores.csv")
```



```

# Análisis completo del sistema

analisis_completo <- datos %>%

# Filtrar servidores activos (excluir mantenimiento, etc.)

filter(estado == "activo") %>%

# Calcular métricas adicionales

mutate( # Calcular una métrica compuesta de carga
  carga_total = cpu_uso _ 0.6 + mem_uso _ 0.4, # CPU pesa 60%, memoria 40%

  # Categorizar nivel de rendimiento basado en carga total
  nivel_rendimiento = case_when(
    carga_total < 50 ~ "Óptimo",          # Carga baja → óptimo
    carga_total >= 50 & carga_total < 70 ~ "Aceptable", # Carga media →
acceptable
    carga_total >= 70 ~ "Crítico"         # Carga alta → crítico
  ),

  # Calcular si está cerca del límite (uso > 80% en cualquier métrica)
  cerca_limite = cpu_uso > 80 | mem_uso > 80 | temp > 70

) %>%

# Agrupar por nivel de rendimiento

group_by(nivel_rendimiento) %>%

# Calcular estadísticas para cada grupo

summarize(
  n_servidores = n(), # Número de servidores en cada categoría
  cpu_promedio = mean(cpu_uso), # Promedio de CPU
  mem_promedio = mean(mem_uso), # Promedio de memoria
  temp_promedio = mean(temp), # Promedio de temperatura
  servidores_cerca_limite = sum(cerca_limite), # Número cerca del límite
  porcentaje_cerca_limite = mean(cerca_limite) \* 100, # Porcentaje cerca del límite
  .groups = "drop" # Eliminar agrupamiento después del summarize
) %>%

# Ordenar por nivel de rendimiento

arrange(factor(nivel_rendimiento, levels = c("Óptimo", "Aceptable", "Crítico")))

print("Análisis completo del rendimiento del sistema:")
print(analisis_completo)

# Guardar el análisis en un archivo CSV

write_csv(analisis_completo, "analisis_rendimiento_completo.csv")
print("Análisis guardado en 'analisis_rendimiento_completo.csv'")

# Crear un resumen ejecutivo

resumen_ejecutivo <- analisis_completo %>%

```

```

summarise(
  total_servidores = sum(n_servidores),
  porcentaje_optimo = sum(n_servidores[nivel_rendimiento == "Óptimo"]) /
  total_servidores _ 100,
  porcentaje_aceptable = sum(n_servidores[nivel_rendimiento == "Aceptable"]) /
  total_servidores _ 100,
  porcentaje_critico = sum(n_servidores[nivel_rendimiento == "Crítico"]) /
  total_servidores _ 100,
  porcentaje_total_cerca_limite = sum(servidores_cerca_limite) / total_servidores _
  100
)

print("Resumen ejecutivo del estado del sistema:")
print(resumen_ejecutivo)

```

3.8 Ejercicio Avanzado: Simulación de Datos y Análisis

Para practicar con datos más realistas, vamos a simular un conjunto de datos más completo.

```

# Función para simular datos de servidores más realistas

simular_datos_servidores <- function(n_servidores = 10, n_mediciones = 24) {
  datos_simulados <- data.frame()

  for (i in 1:n_servidores) { # Determinar tipo de servidor
    tipo <- sample(c("web", "db", "cache", "app"), 1)
    servidor_id <- paste0(tipo, sprintf("%02d", i))

    # Generar características base según tipo
    if (tipo == "web") {
      cpu_base <- sample(40:60, 1)
      mem_base <- sample(50:70, 1)
    } else if (tipo == "db") {
      cpu_base <- sample(30:50, 1)
      mem_base <- sample(60:80, 1)
    } else if (tipo == "cache") {
      cpu_base <- sample(50:70, 1)
      mem_base <- sample(40:60, 1)
    } else { # app
      cpu_base <- sample(45:65, 1)
      mem_base <- sample(55:75, 1)
    }

    for (j in 1:n_mediciones) {
      # Simular variación a lo largo del tiempo
      hora <- j

```

```

variacion <- sin(j / 6) * 15 # Variación sinusoidal simulando patrón diario

cpu <- max(5, min(95, cpu_base + variacion + rnorm(1, 0, 5)))
mem <- max(10, min(90, mem_base + variacion * 0.8 + rnorm(1, 0, 4)))
temp <- 60 + cpu / 2 + rnorm(1, 0, 2)

# Determinar estado basado en métricas
if (cpu > 85 | mem > 85 | temp > 75) {
  estado <- "alerta"
} else if (cpu > 70 | mem > 75 | temp > 70) {
  estado <- "advertencia"
} else {
  estado <- "normal"
}

# Crear fila de datos
fila <- data.frame(
  servidor = servidor_id,
  tipo = tipo,
  hora = hora,
  cpu_uso = round(cpu),
  mem_uso = round(mem),
  temp = round(temp, 1),
  estado = estado
)

datos_simulados <- bind_rows(datos_simulados, fila)
}

return(datos_simulados)
}

# Generar datos simulados

set.seed(123) # Para resultados reproducibles
datos_simulados <- simular_datos_servidores(8, 12) # 8 servidores, 12 mediciones

print("Datos simulados de servidores:")
glimpse(datos_simulados)
head(datos_simulados)

# Análisis de tendencias por tipo de servidor

analisis_tendencias <- datos_simulados %>%
  group_by(tipo, hora) %>%
  summarize(
    n_servidores = n(),
    cpu_promedio = mean(cpu_uso),
    mem_promedio = mean(mem_uso),
    temp_promedio = mean(temp),
    alertas = sum(estado == "alerta"),
    advertencias = sum(estado == "advertencia"),
    .groups = "drop"
  ) %>%
  mutate(

```

```

porcentaje_alertas = (alertas / n_servidores) _ 100,
porcentaje_problemas = ((alertas + advertencias) / n_servidores) _ 100
)

print("Tendencias de rendimiento por tipo de servidor:")
print(analisis_tendencias)

# Identificar horas críticas

horas_criticas <- analisis_tendencias %>%
filter(porcentaje_problemas > 30) %>%
arrange(desc(porcentaje_problemas))

print("Horas con más del 30% de servidores con problemas:")
print(horas_criticas)

# Guardar datos simulados y análisis

write_csv(datos_simulados, "datos_servidores_simulados.csv")
write_csv(analisis_tendencias, "analisis_tendencias.csv")
write_csv(horas_criticas, "horas_criticas.csv")

print("Datos simulados guardados en 'datos_servidores_simulados.csv'")
print("Análisis de tendencias guardado en 'analisis_tendencias.csv'")
print("Horas críticas guardadas en 'horas_criticas.csv'")

```

3.9 Buenas Prácticas y Consejos

1. Manejo de datos faltantes (NA)

```

# Identificar valores faltantes

valores_faltantes <- datos_servidores %>%
summarise(across(everything(), ~sum(is.na(.))))

print("Valores faltantes por columna:")
print(valores_faltantes)

# Estrategias para manejar NA

datos_sin_na <- datos_servidores %>%
drop_na() # Eliminar filas con NA

# O reemplazar NA con valores específicos

datos_reemplazados <- datos_servidores %>%
mutate(
cpu_uso = ifelse(is.na(cpu_uso), mean(cpu_uso, na.rm = TRUE), cpu_uso),

```

```
mem_uso = ifelse(is.na(mem_uso), median(mem_uso, na.rm = TRUE), mem_uso)
)
```

2. Validación de datos

```
# Verificar que los datos están dentro de rangos esperados

validacion <- datos_servidores %>%
  summarise(
    cpu_fuera_rango = sum(cpu_uso < 0 | cpu_uso > 100, na.rm = TRUE),
    mem_fuera_rango = sum(mem_uso < 0 | mem_uso > 100, na.rm = TRUE),
    temp_fuera_rango = sum(temp < 0 | temp > 100, na.rm = TRUE)
  )

print("Validación de rangos de datos:")
print(validacion)
```

3. Optimización para grandes volúmenes de datos

```
# Para datasets muy grandes, considera:

# 1. Usar data.table en lugar de dplyr

# 2. Usar las funciones \_if o \_at de dplyr para operaciones selectivas

# 3. Processar datos en chunks

# Ejemplo con mutate_if

datos_numericos_normalizados <- datos_servidores %>%
  mutate_if(is.numeric, scale) # Normalizar todas las columnas numéricas

print("Datos numéricos normalizados:")
print(datos_numericos_normalizados)
```

3.10 Recursos Adicionales

- Documentación oficial de dplyr: [Guía completa con ejemplos](#)
- Documentación oficial de tidyr: [Referencia de todas las funciones](#)
- Cheatsheet de data manipulation: [Hoja de referencia rápida](#)
- Libro "R for Data Science": [Capítulos 5, 9, 12 y 13 cubren estos temas en profundidad](#)

- Video tutoriales: [Tutorial interactivo](#)

Continuará...